



Same Parent Domain Authentication Flow

A practical integration document for connecting new Next.js/static projects to the existing AtomX access portal, Google login, workspace selection, and selected-session API flow.

PORTAL portal.atomx.in	PROJECT DOMAINS *.atomx.in	API dapi-uat.atomx.in
---------------------------	-------------------------------	--------------------------

Core decision: all projects live under the same parent domain, for example portal.atomx.in , dashboard.atomx.in , tag-series.atomx.in , and new-project.atomx.in . This allows the backend to use parent-domain cookies for the selected workspace session.

1. Objective

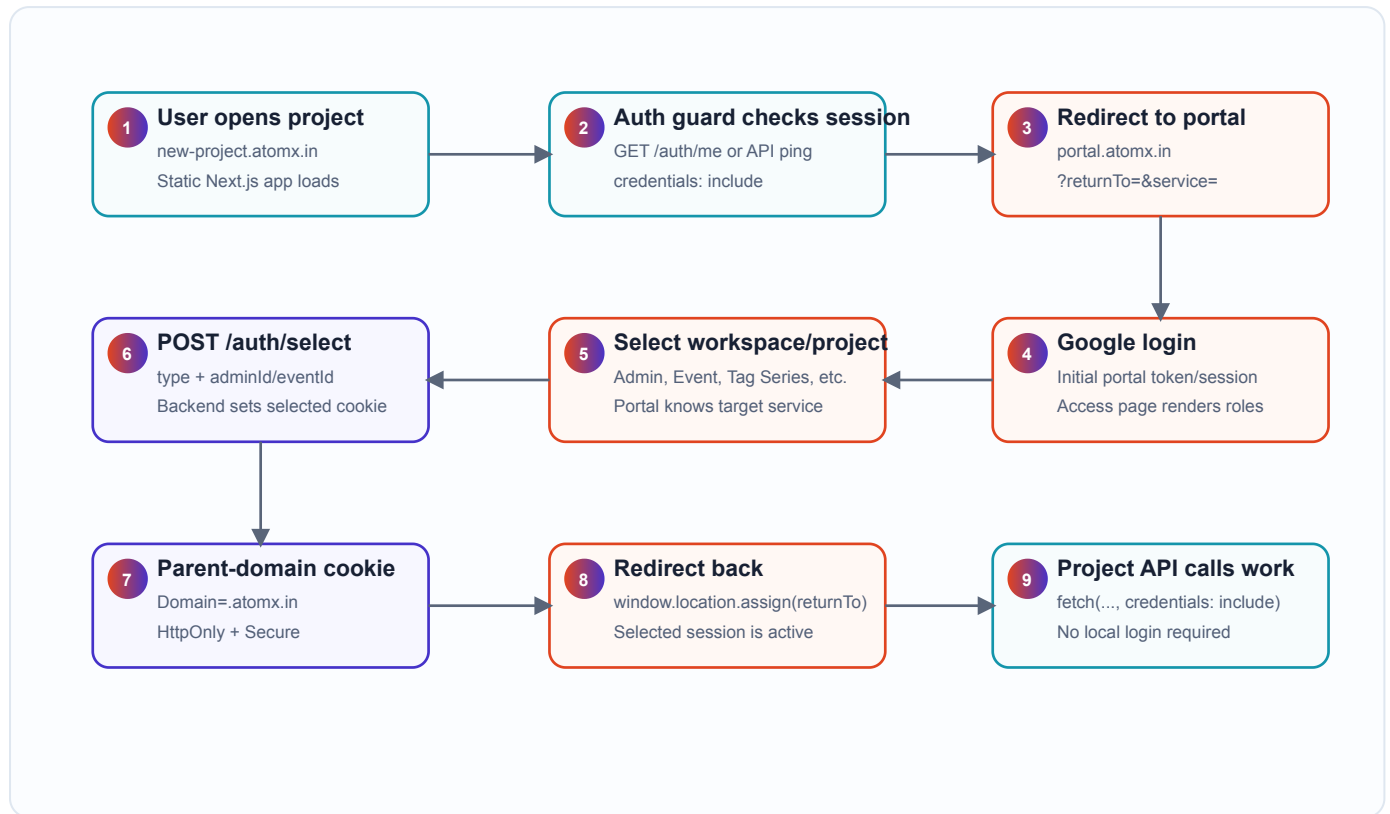
The goal is to let any AtomX project use the same login and access-selection process without rebuilding Google login in every project. A user can open any project subdomain, be redirected to portal.atomx.in for authentication, select the correct workspace/project, and return to the project with an active selected session.

The new project can be created by another worker, built as an out/ static folder, and hosted on a different subdomain, as long as the parent domain remains atomx.in .

2. Domain Model

SERVICE	EXAMPLE	RESPONSIBILITY
Access Portal	portal.atomx.in	Google login, access page, workspace/project selection, /auth/select , redirect back to project.
Dashboard	dashboard.atomx.in	Admin/event list, configuration, operations, vendor/stall/device/menu screens.
New Project	new-project.atomx.in	Own UI and business logic; no Google login implementation; uses AtomX session and API client.
Backend API	dapi-uat.atomx.in	Google auth callback, /auth/select , session cookie, protected APIs.

3. Full Flow Diagram



4. Token and Cookie Flow

There are two different auth moments: the initial login session and the selected workspace/project session. The selected session is the one that should authorize project APIs.

1. **Google login:** user authenticates once through `portal.atomx.in`.
2. **Initial token/session:** backend lets the portal render the access page and available roles.
3. **Workspace selection:** portal sends `POST /auth/select` with the selected role context.
4. **Selected session:** backend updates the session cookie to contain selected context such as `adminId`, `eventId`, or `type`.
5. **Project APIs:** the destination project sends API calls with `credentials: "include"`; the backend reads the selected cookie.

Important: if the cookie is `HttpOnly`, frontend code cannot read it with `document.cookie`. That is good. The project should validate login by calling a session-check endpoint like `/auth/me`, not by reading the cookie directly.

Required Backend Cookie Settings

```
Domain=.atomx.in
Path=/
Secure
HttpOnly
SameSite=None
```

Required CORS Settings

The API must allow credentialed requests from all approved project subdomains. Do not use wildcard CORS with credentials.

```
Allowed origins:
https://portal.atomx.in
https://dashboard.atomx.in
https://tag-series.atomx.in
https://new-project.atomx.in

Allow credentials: true
```

5. Redirect Between Subdomains

A new project should redirect unauthenticated users to the portal with a `returnTo` parameter. The portal should preserve this return URL through login and workspace selection.

Project to Portal

```
https://portal.atomx.in?returnTo=https%3A%2F%2Fnew-project.atomx.in%2F&service=new-project
```

Portal Back to Project

```
https://new-project.atomx.in/
```

Because all apps share `.atomx.in`, the selected cookie set by the backend is available to the project subdomain when the project calls the API.

6. New Project Implementation Guide

Give the following small integration kit to the worker building a new AtomX project. The project does not need Google login code. It only needs an auth guard and an API helper.

FILE	WHERE TO PLACE	PURPOSE
src/lib/atomxAuthConfig.js	New project	Portal URL, API URL, service key, login route settings.
src/lib/atomxApiClient.js	New project	Central API helper using <code>credentials: "include"</code> .
src/components/AtomxAuthGuard.jsx	New project	Client-side session check and redirect to portal.
src/app/layout.jsx or pages/_app.jsx	New project	Wrap the application with <code>AtomxAuthGuard</code> .

7. Code: Auth Config

```
// src/lib/atomxAuthConfig.js
export const ATOMX_AUTH = {
  portalUrl: "https://portal.atomx.in",
  apiBaseUrl: "https://dapi-uat.atomx.in",
  service: "new-project",
  sessionCheckPath: "/auth/me"
};
```

8. Code: API Client

```
// src/lib/atomxApiClient.js
import { ATOMX_AUTH } from "../atomxAuthConfig";

export async function atomxFetch(path, options = {}) {
  const url = path.startsWith("http")
    ? path
    : `${ATOMX_AUTH.apiBaseUrl}${path}`;

  const response = await fetch(url, {
    ...options,
    credentials: "include",
    headers: {
      ...(options.headers || {})
    }
  });

  if (!response.ok) {
    const text = await response.text().catch(() => response.statusText);
    throw new Error(text || `Request failed (${response.status})`);
  }

  return response.json();
}
```

9. Code: Redirect Helper

```
// src/lib/atomxRedirect.js
import { ATOMX_AUTH } from "../atomxAuthConfig";

export function redirectToAtomxPortal() {
  if (typeof window === "undefined") return;

  const url = new URL(ATOMX_AUTH.portalUrl);
  url.searchParams.set("returnTo", window.location.href);
  url.searchParams.set("service", ATOMX_AUTH.service);

  window.location.assign(url.toString());
}
```

10. Code: Auth Guard

```
// src/components/AtomxAuthGuard.jsx
"use client";

import { useEffect, useState } from "react";
import { ATOMX_AUTH } from "../lib/atomxAuthConfig";
import { atomxFetch } from "../lib/atomxAIClient";
import { redirectToAtomxPortal } from "../lib/atomxRedirect";

export default function AtomxAuthGuard({ children }) {
  const [status, setStatus] = useState("checking");

  useEffect(() => {
    let active = true;

    async function checkSession() {
      try {
        await atomxFetch(ATOMX_AUTH.sessionCheckPath);
        if (active) setStatus("ready");
      } catch {
        if (!active) return;
        setStatus("redirecting");
        redirectToAtomxPortal();
      }
    }

    checkSession();
    return () => {
      active = false;
    };
  }, []);

  if (status !== "ready") {
    return null; // or show AtomX loader
  }

  return children;
}
```

11. Where to Use the Guard

Next.js App Router

```
// src/app/layout.jsx
import AtomxAuthGuard from "../components/AtomxAuthGuard";

export default function RootLayout({ children }) {
  return (
    <html lang="en">
      <body>
        <AtomxAuthGuard>
          {children}
        </AtomxAuthGuard>
      </body>
    </html>
  );
}
```

Next.js Pages Router

```
// src/pages/_app.jsx
import AtomxAuthGuard from "../components/AtomxAuthGuard";

export default function App({ Component, pageProps }) {
  return (
    <AtomxAuthGuard>
      <Component {...pageProps} />
    </AtomxAuthGuard>
  );
}
```

12. Access Portal Responsibilities

The portal must accept `returnTo` and `service` in the URL, complete login if needed, then perform workspace selection.

1. Read `returnTo` from URL and sanitize it.
2. Run Google login if there is no active portal session.
3. Show access/workspace cards from the decoded roles.
4. When user selects the target project/workspace, call `POST /auth/select`.
5. Backend sets selected session cookie on `.atomx.in`.
6. Redirect to `returnTo`.

Portal Selection Request

```
POST https://dapi-uat.atomx.in/auth/select
credentials: include
body:
{
  "type": "cashless",
  "eventId": 1840
}

or:
{
  "type": "admin",
  "adminId": 45
}
```


13. API Call Pattern Inside New Project

All project API calls must use the shared helper or at least include `credentials: "include"`.

```
import { atomxFetch } from "../lib/atomxAIClient";

export async function loadEvents() {
  return atomxFetch("/v1/Events/List");
}

export async function loadStalls(eventId) {
  return atomxFetch(`/v1/Stalls/List/Eventwise/${eventId}`);
}
```

Do not build Google login in the new project. The new project should only check session, redirect to portal when needed, and use AtomX APIs with credentials.

14. Static Export Notes

If the new project is exported as a static `out/` folder, the auth guard must run on the client side. This is expected. The static app cannot know the session at build time.

- Use a client component for auth check.
- Do not call protected APIs during static generation.
- Show a loader while checking session.
- If unauthorized, redirect to `portal.atomx.in`.
- After returning, API calls work through the parent-domain cookie.

15. Security Checklist

- Use `HttpOnly` cookies for real session tokens.
- Use `Secure` cookies in production.
- Use `SameSite=None` for cross-subdomain credential requests when needed.
- Set cookie `Domain=.atomx.in`.
- Sanitize `returnTo`; allow only approved `*.atomx.in` destinations.
- Do not put long-lived secrets in query params.
- Allow only known project subdomains in backend CORS.
- Clear selected session on logout.

16. Final Worker Handoff Summary

Give this summary to anyone building a new project:

1. Host the app on a subdomain under `atomx.in`.
2. Do not implement Google login.
3. Add `AtomxAuthGuard` at the root of the app.
4. In the guard, call `/auth/me` or a session-check API using `credentials: "include"`.
5. If unauthorized, redirect to `portal.atomx.in?returnTo=currentUrl&service=projectKey`.
6. Use `atomxFetch` for all AtomX backend calls.
7. Build the app normally and deploy the `out/` folder to its subdomain.

Document generated for AtomX Portal V1 monorepo integration. Keep this file in sync with backend cookie settings and access portal redirect behavior.